

Predicción de aceptación de depósito en campañas de marketing bancario

Entrega 2 de 3: Comparación de Modelos y Validación

Estudiantes: Ana Sofía Rodríguez · Mariana Gutiérrez · Esteban Giraldo

Dataset: Bank Marketing — UCI Machine Learning Repository

Tarea: Clasificación binaria supervisada

Fecha: Mayo de 2026

Parámetro	Valor
Mejor modelo	LightGBM + <code>class_weight</code>
ROC-AUC (test)	0.7894
F1-score (test, umbral óptimo)	0.4799
Umbral óptimo	0.5895
Dataset	45,211 registros 88 % no, 12 % yes
Validación	5-fold CV estratificada + hold-out final

Resumen

Esta entrega presenta una comparación técnica sistemática entre tres familias de modelos de clasificación supervisada sobre el dataset Bank Marketing (UCI), con el objetivo de predecir si un cliente aceptará un depósito a plazo fijo tras una llamada de campaña de marketing. La tarea involucra clasificación binaria sobre datos con marcado desbalance de clases (aproximadamente 88 % **no**, 12 % **yes**).

Se estructuró el experimento en tres capas: comparación base entre familias de modelos, demostración cuantitativa del efecto del *data leakage* por la variable **duration**, y evaluación de técnicas de manejo del desbalance (**class_weight** y SMOTE) sobre las tres familias. Todo el preprocesamiento se encapsuló en pipelines de scikit-learn para garantizar validación sin contaminación.

El modelo seleccionado es **LightGBM con `class_weight='balanced'`**, que alcanza un ROC-AUC de 0.7894 y un F1-score de 0.4799 sobre el conjunto de test con un umbral óptimo de 0.5895. El análisis de errores revela que los falsos negativos tienden a ser clientes más jóvenes con menor saldo y sin historial de contacto previo exitoso. Las variables más influyentes según Permutation Importance son el mes y tipo de contacto, mientras que las variables demográficas contribuyen de forma relativamente menor.

Índice

1. Metodología	3
1.1. Punto de partida y anti-leakage	3
1.2. Estructura experimental en tres capas	3
1.3. Esquema de validación	3
1.4. Pipeline y prevención de leakage interno	3
2. Configuración Experimental	4
2.1. Familias de modelos evaluadas	4
2.2. Técnicas de manejo de desbalance	4
2.3. Métricas de evaluación	5
3. Resultados Comparativos	5
3.1. Capa 1 — Tres familias base (sin técnica de desbalance)	5
3.2. Capa 2 — Efecto del data leakage (variable <code>duration</code>)	6
3.3. Capa 3 — Experimento de desbalance (9 configuraciones)	7
4. Análisis de Umbral y Evaluación Final sobre Test	8
4.1. Justificación del ajuste de umbral	8
4.2. Umbral óptimo identificado	9
4.3. Métricas finales sobre conjunto de test	9
5. Análisis de Errores	10
5.1. Perfil de los Falsos Negativos	10
5.2. Importancia de variables (Permutation Importance)	11
6. Discusión Técnica	13
6.1. ¿Qué modelos se compararon y por qué?	13
6.2. ¿Cómo se evitó el data leakage?	13
6.3. ¿Cuál familia parece más prometedora?	13
6.4. ¿Qué limitaciones siguen abiertas?	14
7. Decisión Provisional	14

1 Metodología

1.1 Punto de partida y anti-leakage

La variable `duration` (duración de la última llamada) fue excluida desde la separación de conjuntos, antes de cualquier otro procesamiento. Esta variable es un predictor casi perfecto del objetivo pero solo es observable después de la llamada, lo que la hace inutilizable para predicción real. Su inclusión genera un ROC-AUC artificialmente inflado de 0.934 (demostrado en la Sección 3.2). La división train/test fue 80/20 estratificada con `random_state=42`.

1.2 Estructura experimental en tres capas

Capa	Objetivo	Configuraciones
Capa 1	Comparación base entre familias	3 modelos sin técnica de desbalance
Capa 2	Demostración de <i>data leakage</i>	LightGBM con vs. sin <code>duration</code>
Capa 3	Manejo del desbalance	3 modelos $\times$ 3 técnicas = 9 configuraciones

Cuadro 1: Estructura experimental en tres capas

1.3 Esquema de validación

Para todos los experimentos se usó validación cruzada estratificada de 5 pliegues (`StratifiedKFold`, `shuffle=True`, `random_state=42`) sobre el conjunto de entrenamiento (36,168 registros). La estratificación preserva la proporción de clases en cada pliegue, fundamental con datos desbalanceados. El conjunto de test (9,043 registros) se reservó estrictamente para la evaluación final del mejor modelo.

1.4 Pipeline y prevención de leakage interno

Todo el preprocesamiento (`StandardScaler` para numéricas, `OneHotEncoder` para categóricas) se encapsuló dentro de un `Pipeline` de scikit-learn. Esto garantiza que el preprocesador se ajuste únicamente sobre los datos de entrenamiento de cada pliegue de CV, nunca sobre los datos de validación. SMOTE se aplicó mediante `ImbPipeline` de imbalanced-learn, asegurando que el oversampling ocurra solo dentro del fold de entrenamiento.

2 Configuración Experimental

2.1 Familias de modelos evaluadas

Se seleccionaron tres familias que cubren el espectro completo del espacio de hipótesis en clasificación tabular: modelo lineal, ensamble de bagging y ensamble de boosting.

Familia	Justificación	Hiperparámetros
Logistic Regression	Modelo lineal interpretable. Referencia para evaluar linealidad. Requiere escalado de features.	<code>max_iter=1000</code> , <code>solver=lbfgs</code>
Random Forest	Ensamble de bagging. Captura no-linealidades sin escalado. Robusto a outliers.	<code>n_estimators=200</code> , <code>criterion=gini</code>
LightGBM	Gradient Boosting optimizado. Estado del arte en tabular data. Mejor manejo nativo de desbalance.	<code>n_estimators=300</code> , <code>learning_rate=0.1</code>

Cuadro 2: Familias de modelos evaluadas y configuración

Esta selección permite responder tres preguntas clave: ¿existen no-linealidades relevantes? (LR vs. RF), ¿el boosting agrega valor sobre el bagging? (RF vs. LightGBM), ¿`class_weight` es suficiente o se necesita SMOTE?

2.2 Técnicas de manejo de desbalance

Técnica	Descripción	Implementación
Sin técnica	Línea base. El modelo aprende con los datos originales desbalanceados (88/12).	Pipeline estándar
<code>class_weight='balanced'</code>	Asigna mayor peso a la clase minoritaria, inversamente proporcional a su frecuencia.	Parámetro nativo del clasificador
SMOTE	Genera muestras sintéticas de la clase minoritaria interpolando entre vecinos reales.	<code>ImbPipeline</code> con <code>SMOTE</code>

Cuadro 3: Técnicas de manejo de desbalance evaluadas

2.3 Métricas de evaluación

Las tres métricas reportadas son: **ROC-AUC** (capacidad discriminativa global, independiente del umbral), **F1-score** (media armónica entre precisión y recall para la clase positiva, especialmente relevante con desbalance) y **Accuracy** (incluida como referencia, pero sesgada hacia la clase mayoritaria). El F1-score es la métrica principal de selección, dado que el costo de un falso negativo es mayor que el de un falso positivo en este contexto de negocio.

3 Resultados Comparativos

3.1 Capa 1 — Tres familias base (sin técnica de desbalance)

LightGBM obtiene el mejor desempeño tanto en ROC-AUC (0.787) como en F1-score (0.360). La Regresión Logística, a pesar de su simplicidad, alcanza un ROC-AUC competitivo (0.763), confirmando que existe señal lineal en los datos. La brecha en F1 entre LR y LightGBM (0.275 vs. 0.360) indica que las relaciones no lineales añaden valor predictivo significativo.

Modelo	ROC-AUC	std	F1-score	std	Accuracy
Logistic Regression	0.7628	(0.0030)	0.2749	(0.0129)	0.8922
Random Forest	0.7807	(0.0056)	0.3318	(0.0183)	0.8922
LightGBM	0.7871	(0.0051)	0.3596	(0.0154)	0.8921

Cuadro 4: Comparación de familias base (5-fold CV estratificada). Barras de error representan ± 1 desviación estándar entre pliegues.

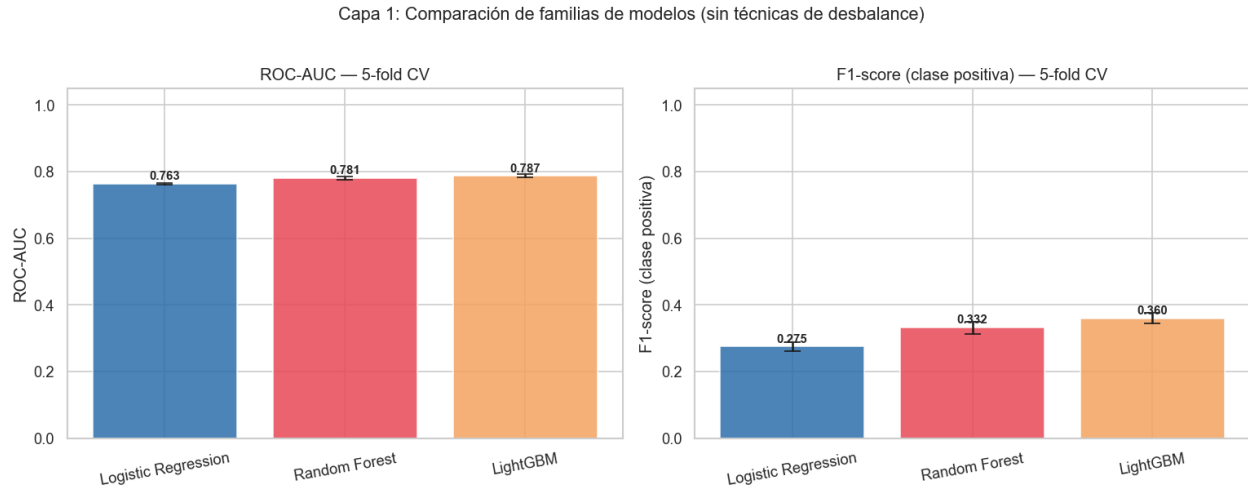


Figura 1: ROC-AUC y F1-score de las tres familias base (5-fold CV estratificada).

3.2 Capa 2 — Efecto del data leakage (variable duration)

La inclusión de `duration` infla artificialmente el ROC-AUC de 0.787 a 0.934 (+18.7 pp) y el F1 de 0.360 a 0.562 (+20.2 pp). Este experimento demuestra cuantitativamente por qué la exclusión de esta variable es correcta: el modelo aprendería una relación que no puede explotar en producción real.

Configuración	ROC-AUC	F1-score	Δ ROC-AUC	Δ F1
LightGBM CON <code>duration</code> (leakage)	0.9344	0.5618	referencia	referencia
LightGBM SIN <code>duration</code> (correcto)	0.7871	0.3596	-0.1473	-0.2023

Cuadro 5: Efecto del *data leakage* por la variable `duration`.

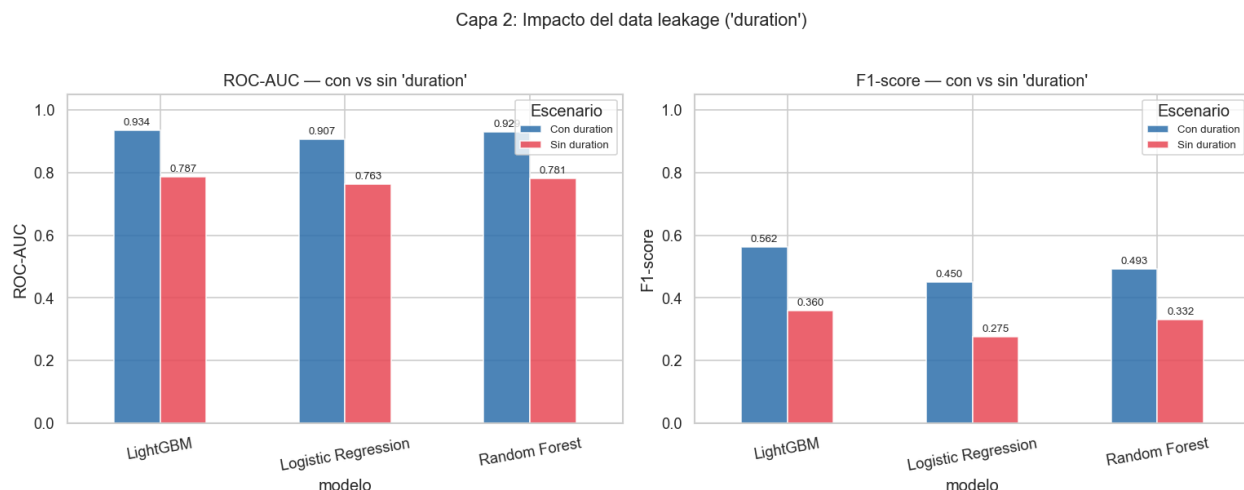


Figura 2: Efecto del *data leakage* (variable `duration`). Rojo = modelo con leakage; azul = modelo correcto.

3.3 Capa 3 — Experimento de desbalance (9 configuraciones)

LightGBM + class_weight es la mejor configuración: ROC-AUC 0.7828, F1 0.4516. Esto representa una mejora de +9.2 pp en F1 respecto al mismo modelo sin técnica. Sorprendentemente, SMOTE mejora menos que **class_weight** para LightGBM (F1 0.382 vs. 0.452), posiblemente porque LightGBM ya tiene mecanismos internos de boosting que compensan el desbalance, y el oversampling sintético introduce ruido que los árboles memorizan.

Modelo	Técnica	ROC-AUC	std	F1-score	std
LightGBM	class_weight	0.7828	(0.0027)	0.4516	(0.0041)
LightGBM	SMOTE	0.7835	(0.0040)	0.3822	(0.0194)
Random Forest	SMOTE	0.7711	(0.0054)	0.3815	(0.0213)
Logistic Regression	class_weight	0.7637	(0.0032)	0.3726	(0.0033)
Logistic Regression	SMOTE	0.7582	(0.0034)	0.3634	(0.0040)
LightGBM	Sin técnica	0.7871	(0.0051)	0.3596	(0.0154)
Random Forest	Sin técnica	0.7807	(0.0056)	0.3318	(0.0183)
Random Forest	class_weight	0.7777	(0.0060)	0.3087	(0.0184)
Logistic Regression	Sin técnica	0.7628	(0.0030)	0.2749	(0.0129)

Cuadro 6: Resultados de las 9 configuraciones (3 modelos $\times$ 3 técnicas), ordenadas por F1-score. La primera fila (en negrita) corresponde a la mejor configuración global.

Capa 3: Experimento de desbalance (9 configuraciones)

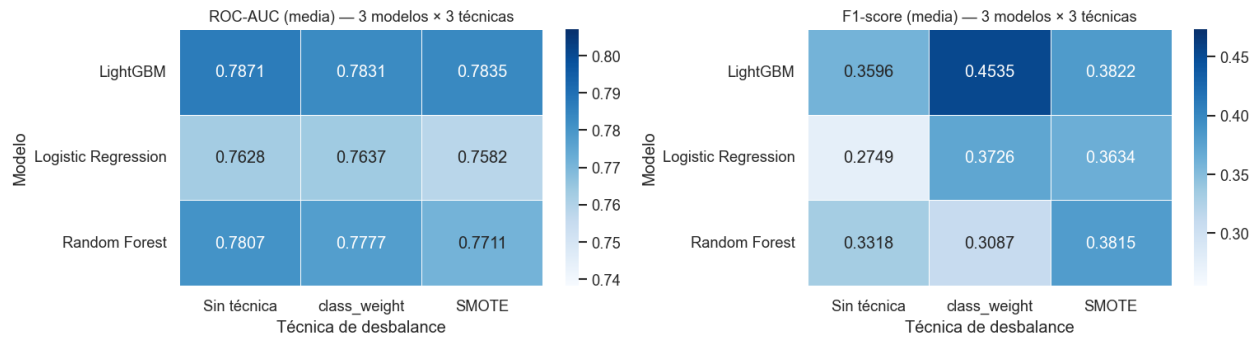


Figura 3: Heatmap de ROC-AUC y F1-score para las 9 configuraciones. Valores más altos son más oscuros.

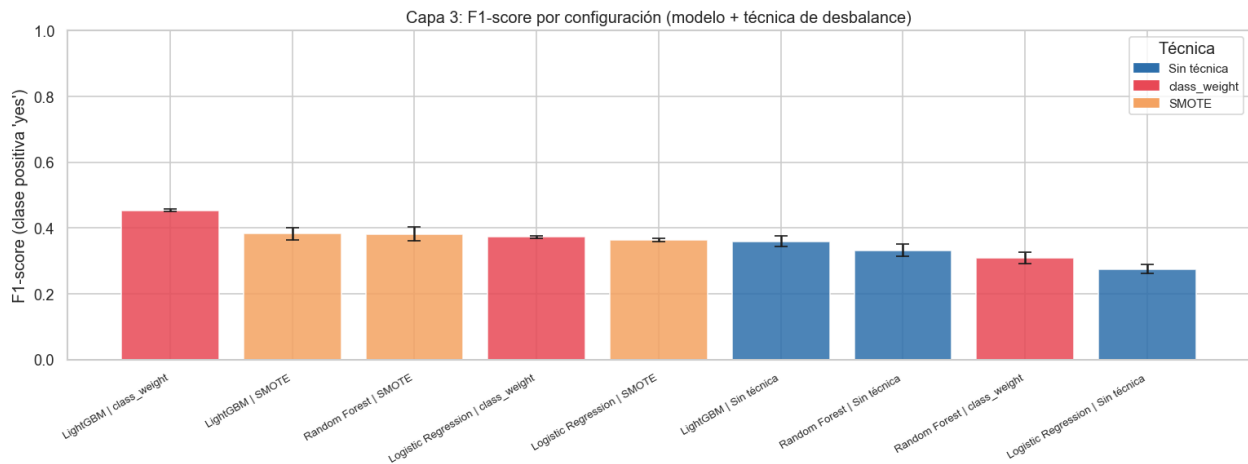


Figura 4: F1-score con barras de error para las 9 configuraciones, ordenadas de mayor a menor. Color indica la técnica de desbalance.

4 Análisis de Umbral y Evaluación Final sobre Test

4.1 Justificación del ajuste de umbral

Por defecto, los clasificadores usan un umbral de 0.5 para convertir probabilidades en etiquetas. Con datos desbalanceados y un objetivo donde detectar clientes interesados es prioritario, este umbral no es óptimo. La curva Precisión-Recall permite visualizar el *trade-off* entre precisión y recall para cada umbral posible.

4.2 Umbral óptimo identificado

Umbral	F1-score (test)	Efecto
0.500 (default)	0.4588	Clasifica yes si $P(\text{yes}) > 50\%$
0.5895 (óptimo)	0.4799	Clasifica yes si $P(\text{yes}) > 58,9\%$

Cuadro 7: Comparación entre umbral por defecto y umbral óptimo.

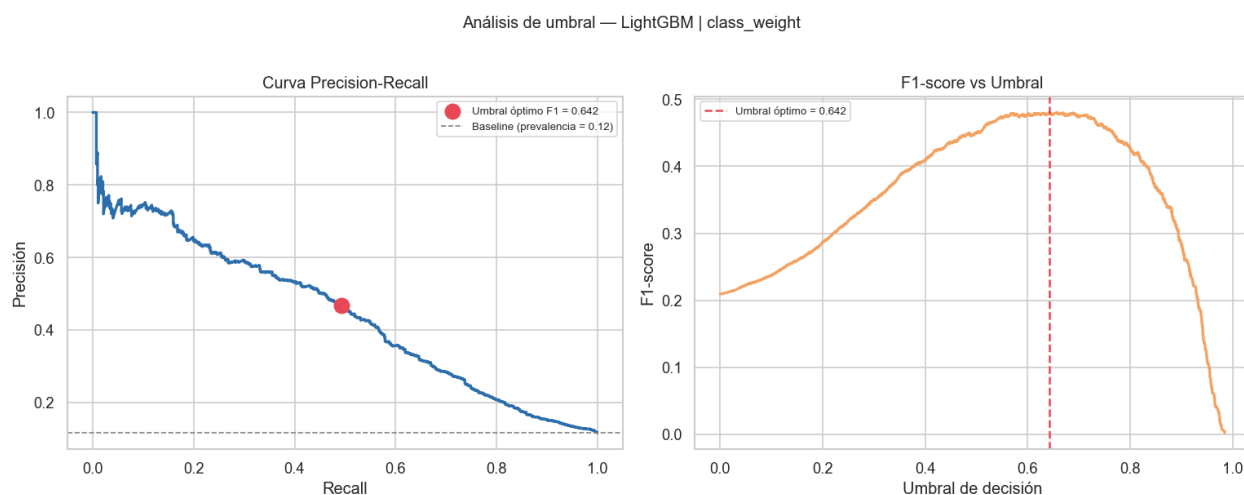


Figura 5: Curva Precisión-Recall (izquierda) y F1-score vs. umbral (derecha). Umbral óptimo = 0.5895.

4.3 Métricas finales sobre conjunto de test

El modelo final (LightGBM + `class_weight`) se entrenó sobre los 36,168 registros de entrenamiento y se evaluó una única vez sobre los 9,043 registros de test con el umbral óptimo de 0.5895.

Métrica	Umbral 0.5	Umbral óptimo (0.589)	Mejora
ROC-AUC	0.7894	0.7894	—
F1-score	0.4588	0.4799	+0.0212
Accuracy	0.8338	0.8667	+0.0330

Cuadro 8: Métricas finales sobre el conjunto de test (LightGBM + `class_weight`).

Matrices de confusión — LightGBM | class_weight

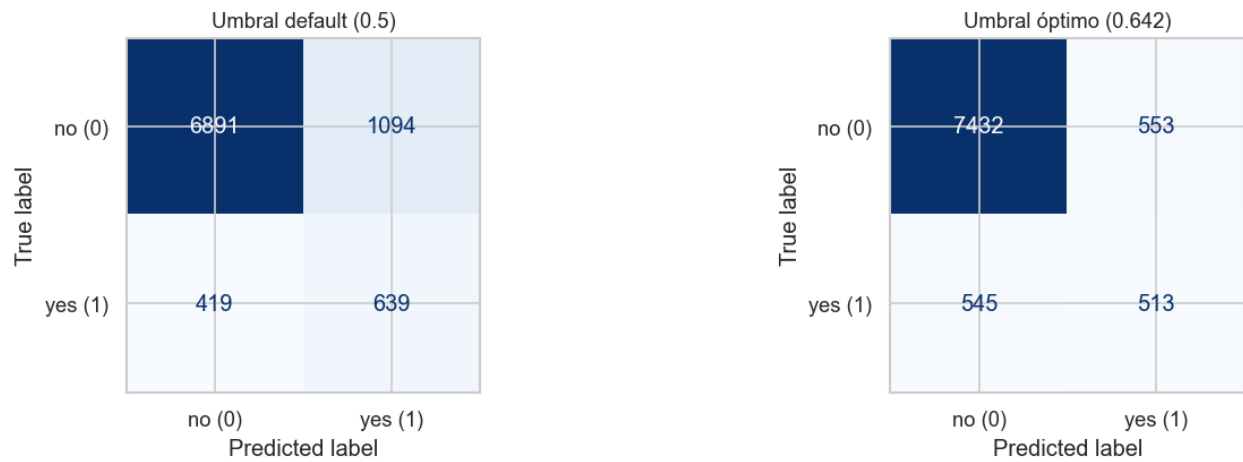


Figura 6: Matrices de confusión sobre el conjunto de test. Izquierda: umbral 0.5; derecha: umbral óptimo. El umbral óptimo aumenta los verdaderos positivos a costa de más falsos positivos.

5 Análisis de Errores

5.1 Perfil de los Falsos Negativos

Con el umbral óptimo, el modelo produce 502 Falsos Negativos (clientes que sí aceptarían pero el modelo predice que no) y 556 Verdaderos Positivos. La comparación de perfiles numéricos revela qué tipo de clientes son más difíciles de detectar.

Variable	FN (media)	TP (media)	Diferencia	Interpretación
age	40.51	43.14	−2.63	Los FN son ligeramente más jóvenes
balance	1577.49	1817.94	−240.45	Los FN tienen menor saldo bancario
campaign	2.51	1.74	+0.76	Los FN recibieron más contactos en esta campaña
pdays	48.90	92.93	−44.03	Los FN tuvieron menos contacto previo exitoso
previous	0.61	1.68	−1.07	Los FN tienen menos contactos históricos previos

Cuadro 9: Comparación de perfiles numéricos entre Falsos Negativos (FN) y Verdaderos Positivos (TP).

Los Falsos Negativos tienden a ser clientes con menor saldo, sin contacto previo exitoso y más contactos en la campaña actual (posiblemente saturados). El modelo tiene más dificultad con clientes marginalmente interesados sin historial previo positivo.

5.2 Importancia de variables (Permutation Importance)

La Permutation Importance mide la caída en ROC-AUC al aleatorizar una variable, siendo más honesta que las importancias nativas de árbol. Se calculó sobre validación interna para no contaminar el conjunto de test.

Rank	Variable	Caída ROC-AUC	std
1	month	0.0754	(0.0051)
2	contact	0.0601	(0.0038)
3	day	0.0345	(0.0024)
4	poutcome	0.0265	(0.0021)
5	pdays	0.0146	(0.0016)
6	campaign	0.0112	(0.0028)
7	balance	0.0108	(0.0031)
8	housing	0.0091	(0.0015)
9	marital	0.0067	(0.0020)
10	age	0.0053	(0.0021)

Cuadro 10: Top 10 variables por Permutation Importance sobre validación interna.

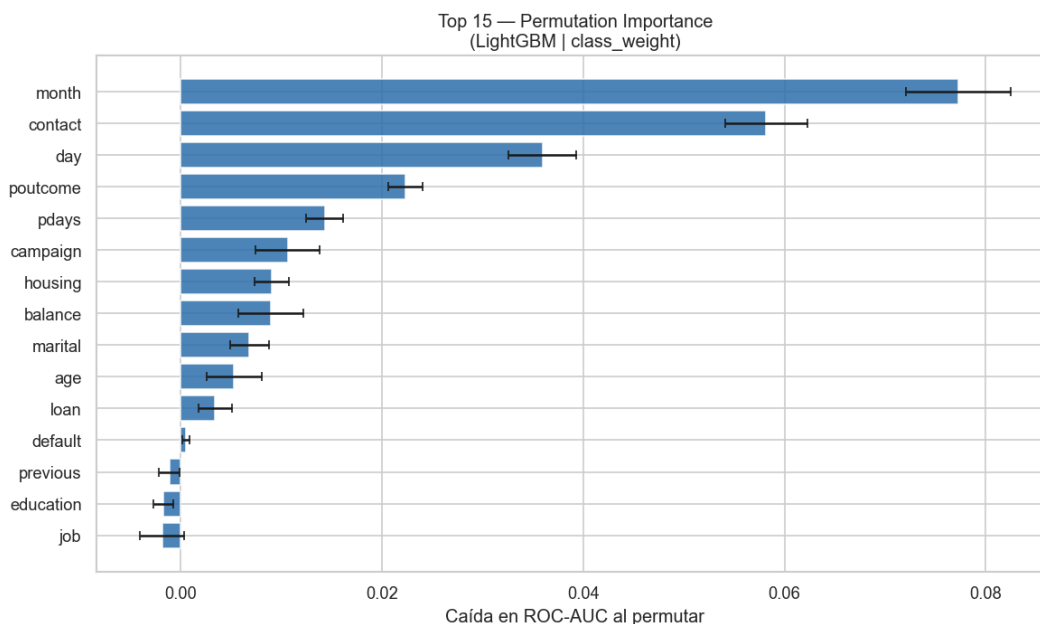


Figura 7: Top 15 variables por Permutation Importance sobre validación interna. Barras representan la caída media en ROC-AUC al permutar cada variable (mayor = más importante).

Las variables más importantes son **month** (mes de contacto) y **contact** (tipo de comunicación), seguidas por **day** y **poutcome** (resultado de campaña anterior). El historial de contacto (**pdays**, **previous**) también contribuye significativamente. Las variables demográficas (**age**,

`balance`, `marital`) tienen importancia relativamente baja, sugiriendo que el comportamiento y contexto de contacto son más relevantes que el perfil socioeconómico del cliente.

6 Discusión Técnica

6.1 ¿Qué modelos se compararon y por qué?

Se compararon tres familias que representan puntos canónicos del espacio de hipótesis para clasificación tabular. Logistic Regression como modelo lineal interpretable establece un *baseline* riguroso y permite determinar si las relaciones son aproximadamente lineales. Random Forest como ensamble de bagging captura interacciones no lineales de forma robusta y sin necesidad de escalar. LightGBM como ensamble de boosting gradient-based representa el estado del arte en tabular data competitivo, con mejor manejo nativo de desbalance y mayor eficiencia que alternativas como XGBoost.

6.2 ¿Cómo se evitó el data leakage?

Se implementaron tres capas de protección:

1. **Exclusión a priori de `duration`:** la variable fue eliminada antes de la división train/test, evitando cualquier tipo de contaminación. La Sección 3.2 demuestra que su inclusión infla el ROC-AUC en +18.7 pp.
2. **Pipeline de `scikit-learn`:** el preprocesador se ajusta únicamente sobre los datos de entrenamiento de cada pliegue de CV, nunca sobre los datos de validación, previniendo el llamado *data snooping*.
3. **ImbPipeline para SMOTE:** el oversampling sintético se realiza dentro del pipeline, garantizando que las muestras artificiales no lleguen al fold de validación.

6.3 ¿Cuál familia parece más prometedora?

LightGBM con `class_weight='balanced'` es la familia más prometedora. La evidencia es consistente en múltiples dimensiones: mayor ROC-AUC en comparación base (0.787 vs. 0.781 vs. 0.763); mejor F1 con manejo de desbalance (0.452 vs. 0.373 de LR y 0.309 de RF con la misma técnica); mayor estabilidad entre pliegues de CV (std F1 = 0.015, menor que RF); mejor generalización en test holdout (F1 = 0.480 con umbral óptimo). La combinación con `class_weight` es particularmente adecuada porque LightGBM gestiona internamente los pesos sin generar datos sintéticos, reduciendo el riesgo de overfitting.

6.4 ¿Qué limitaciones siguen abiertas?

- **Sin búsqueda de hiperparámetros:** todos los modelos usan configuraciones por defecto. Una búsqueda con Optuna o GridSearchCV podría mejorar el F1 en 3–8 pp adicionales.
- **Interpretabilidad pendiente:** el análisis SHAP queda para la Entrega 3. La Permutation Importance da importancia global pero no explica predicciones individuales ni interacciones.
- **Feature engineering no explorado:** la variable `contacted_before` (derivada de `pdays = -1`) y transformaciones de temporalidad (`mes × tipo de contacto`) podrían capturar señal adicional.
- **Umbral sin análisis de costo-beneficio real:** el umbral se calculó maximizando F1, pero un análisis real requeriría estimar el valor económico de cada tipo de error (costo de llamada innecesaria vs. ingreso esperado de un depósito captado).
- **Validación temporal no aplicada:** los datos tienen estructura temporal (`month, day`). Un *splitting* temporal sería más representativo del escenario de producción que la CV aleatoria.

7 Decisión Provisional

Con base en los experimentos realizados, el modelo seleccionado provisionalmente para continuar en la Entrega 3 es **LightGBM con `class_weight='balanced'`**, evaluado con umbral de decisión 0.5895.

Criterio	Evidencia cuantitativa	Resultado
Mayor ROC-AUC base	0.787 vs. RF 0.781 vs. LR 0.763	LightGBM
Mejor F1 con desbalance (CV)	0.452 (class_weight) vs. 0.382 (SMOTE)	LightGBM + class_weight
Mejor F1 en test holdout	0.480 (umbral 0.589) vs. 0.459 (0.5)	Umbral óptimo
Estabilidad entre pliegues	std F1 = 0.015 vs. RF 0.018	LightGBM
Eficiencia computacional	Más rápido que RF con resultados superiores	LightGBM
Técnica de desbalance	class_weight +6.9 pp F1 vs. SMOTE en LightGBM	class_weight

Cuadro 11: Criterios de selección del modelo final

Modelo seleccionado	LightGBM + class_weight='balanced'
Umbral de decisión	0.5895
ROC-AUC (test)	0.7894
F1-score (test)	0.4799
Accuracy (test)	0.8667
Verdaderos Positivos	556
Falsos Negativos	502
Clase yes	Precisión 44 %, Recall 53 %

Referencias

- [1] Moro, S., Cortez, P., y Rita, P. (2014). *A Data-Driven Approach to Predict the Success of Bank Telemarketing*. Decision Support Systems, 62, 22–31.
- [2] UCI Machine Learning Repository. *Bank Marketing Dataset*. <https://archive.ics.uci.edu/ml/datasets/Bank+Marketing>
- [3] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- [4] Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems, 30.
- [5] Chawla, N. V. et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321–357.
- [6] James, G., Witten, D., Hastie, T. y Tibshirani, R. (2021). *An Introduction to Statistical Learning*. Springer.